

Functional Programming Foundations

5-Day Beginner Programming Course

CANDIDATE EDITION

Table of Contents

1. [Course Overview](#)
2. [Day 1: Data, Variables & Operators](#)
3. [Day 2: Expressions, Statements & Conditions](#)
4. [Day 3: Loops, Lists & Data Structures](#)
5. [Day 4: Functions, Scope & Errors](#)
6. [Day 5: Input/Output, Algorithms & Complete Design](#)
7. [Final Assessment](#)
8. [Course Summary](#)
9. [Glossary](#)

Course Overview

What You Will Learn

By the end of this 5-day course, you will understand:

- What data types exist in programming
- How to name and store data using variables
- How to perform operations on data
- How to ask questions (expressions) and give commands (statements)
- How to make decisions using conditions
- How to repeat actions using loops
- How to organize complex data using lists and records
- How to package logic using functions
- How to control visibility using scope
- How to identify and fix errors
- How to handle input and output

- How to solve problems step-by-step using algorithms
- How to design a complete program from scratch

Course Materials

You will need:

- Notebook or paper
- Pen/pencil
- Index cards (provided)
- Sticky notes (provided)

The Golden Rule of Functional Programming

Everything is data flowing through functions. Variables never change. Expressions do all the work. Statements only at the edges.

Day 1: Data, Variables & Operators

Learning Objectives

By the end of Day 1, you will be able to:

- Identify the 5 basic data types
- Create and use immutable variables
- Perform operations using operators
- Spot and fix type errors

1.1 Data Types

What Are Data Types?

Data types are categories of information. Just like you have different tools for different jobs (hammer for nails, screwdriver for screws), programming has different types for different kinds of data.

The 5 Basic Data Types

1. Numbers

Numbers are exactly what you think they are.

Examples:

5
3.14
-10
1000
0

What you can do with numbers:

- Add (+)
- Subtract (-)
- Multiply (*)
- Divide (/)
- Find remainder (%)

Real-world examples:

- Age (25)
- Price (\$19.99)
- Temperature (72.5)
- Score (87)

2. Text (String)

Text is any letters, words, or sentences. Always written in quotes.

Examples:

"Hello"
"John"
"The sky is blue"
"123" (this is text, not a number!)
"True" (this is text, not a boolean!)

What you can do with text:

- Combine text (+)
- Count letters (.length)
- Convert to uppercase
- Convert to lowercase

Real-world examples:

- Name ("Alice")
- City ("London")
- Email ("alice@email.com")
- Address ("123 Main Street")

3. Booleans

Booleans are true or false. Like a light switch - on or off.

Examples:

```
true  
false
```

What you can do with booleans:

- AND (both must be true)
- OR (at least one must be true)
- NOT (opposite)

Real-world examples:

- Is it raining? (true/false)
- Are you over 18? (true/false)
- Is the light on? (true/false)
- Did you pass? (true/false)

4. Lists (Arrays)

Lists hold multiple items in order.

Examples:

```
[1, 2, 3, 4, 5]  
["apple", "banana", "cherry"]  
[true, false, true]  
[10, "hello", true] (mixed types are allowed but not recommended)  
[] (empty list)
```

What you can do with lists:

- Count items (`.length`)
- Get first item (`[0]`)
- Get last item (`[length-1]`)
- Combine lists (`+`)
- Check if item exists

Real-world examples:

- Shopping cart (`["milk", "eggs", "bread"]`)
- Student grades (`[85, 90, 78, 92]`)
- To-do list (`["wake up", "brush teeth", "eat breakfast"]`)

5. Nothing (Null/Undefined)

Nothing represents "empty" or "missing".

Examples:

```
null
undefined
```

Real-world examples:

- Empty seat on a bus
- Missing answer on a form
- Undefined variable

Quick Reference: Data Types

Type	Example	Use For
Number	25 , 3.14	Age, price, temperature
Text/String	"Hello"	Name, address, message
Boolean	true , false	Yes/no, true/false decisions
List/Array	[1,2,3]	Multiple items in order
Nothing	null	Missing or empty values

1.2 Variables

What Is a Variable?

A **variable** is a name that points to a value.

Example:

```
age = 25
name = "John"
isStudent = true
scores = [85, 90, 78]
```

Instead of writing "25" everywhere, you write "age". Instead of writing [85, 90, 78] , you write "scores".

Variables Are Like Lockers

Think of variables as lockers:

- Locker #42 contains your backpack
- You say "my locker" instead of "the metal box on the third floor"
- The name ("my locker") points to the value (backpack)

The Golden Rule: Variables Never Change

In functional programming, **variables never change**. Once you put a value in a variable, it stays there forever.

WRONG (mutation):

```
age = 25
age = 26    ← NEVER DO THIS! You're erasing history!
```

RIGHT (immutable):

```
age = 25
newAge = 26    ← Create a new variable instead
```

Why Immutability Matters

History: You can always see what the value was before **Predictability:** No surprises - values never change unexpectedly **Safety:** You can always trace back to the original data

Real-world example: Bank accounts don't erase old balances. They create a new balance after each transaction. This is immutability.

1.3 Operators

What Are Operators?

Operators are actions you can perform on data.

Types of Operators

Arithmetic Operators

Used with numbers.

Operator	Meaning	Example	Result
+	Add	5 + 3	8
-	Subtract	10 - 4	6
*	Multiply	3 * 4	12
/	Divide	20 / 5	4
%	Remainder	17 % 5	2

Comparison Operators

Used to compare values. Always return true or false.

Operator	Meaning	Example	Result
>	Greater than	10 > 5	true
<	Less than	3 < 8	true
>=	Greater than or equal	5 >= 5	true
<=	Less than or equal	4 <= 3	false
==	Equal to (value only)	5 == "5"	true
===	Equal to (value and type)	5 === "5"	false
!=	Not equal	10 != 5	true

Important: == checks only value. === checks both value AND type. Always use === unless you have a good reason not to.

Logical Operators

Used with booleans. Combine true/false values.

Operator	Meaning	Example	Result
AND	Both must be true	true AND true	true
AND	Both must be true	true AND false	false
OR	At least one true	true OR false	true
OR	At least one true	false OR false	false
NOT	Opposite	NOT true	false
NOT	Opposite	NOT false	true

Real-world examples:

- "Is it raining AND do I have an umbrella?" → Only true if both are true
- "Is it a weekend OR a holiday?" → True if either is true
- "Is it NOT raining?" → Opposite of raining

Text Operators

Operator	Meaning	Example	Result
+	Combine text	"Hello" + "World"	"HelloWorld"
+	Combine with space	"Hello" + " " + "World"	"Hello World"
.length	Count letters	"Hello".length	5

List Operators

Operator	Meaning	Example	Result
+	Combine lists	[1,2] + [3,4]	[1,2,3,4]
.length	Count items	[1,2,3].length	3
[0]	First item	[5,10,15][0]	5

1.4 Type Errors

What Is a Type Error?

A **type error** happens when you try to use the wrong type of data with an operator.

Common Type Errors

Error 1: Text × Number

```
"Hello" * 3    ← ERROR!
```

Why? You can't multiply text. Text doesn't have a mathematical value.

Error 2: Number + Boolean

```
5 + true    ← ERROR!
```

Why? You can't add a number and a boolean. Booleans are true/false, not numbers.

Error 3: List + Number

```
[1,2] + 3    ← ERROR!
```

Why? You can't add a list and a number. They're different types.

Error 4: Divide by Zero

```
10 / 0    ← ERROR!
```

Why? Dividing by zero is mathematically impossible.

Special Case: Text + Number

```
"5" + 3    → "53" (works!)
```

Why? Text takes over and converts the number to text. This is called **type coercion**.

Rule: Don't mix types unless you know exactly what will happen.

Day 1 Exercises

Exercise 1.1: Sort the Boxes

Instructions: You have been given a bag with mixed items. Sort them into the correct data type boxes.

Items to sort:

```
5
"Hello"
true
[1, 2, 3]
null
3.14
"John"
false
["a", "b", "c"]
undefined
-10
"123"
[true, false]
0
```

Your answers:

Number Box	Text Box	Boolean Box	List Box	Nothing Box

Exercise 1.2: Variable Lockers

Instructions: For each value, create a variable name and write it down. Then "change" the value without erasing the original.

Example:

```
Value: 25
Variable: age = 25
Change to 26:
newAge = 26 ← This is correct! (keeps both)
```

Your turn:

Value	Variable	Change to	New Variable
"John"	name = "John"	"Jane"	
true	isActive = true	false	
[1,2,3]	numbers = [1,2,3]	[1,2,3,4]	
100	price = 100	90	
"London"	city = "London"	"Paris"	

Exercise 1.3: Calculator Game

Instructions: Solve these expressions without a calculator. Show your work.

Expression	Answer	Show Your Work
$5 + 3 \times 2$		
"Hello" + " " + "World"		
$10 > 5$ AND true		
$[1,2] + [3,4]$		
$15 \% 4$		
"Apple".length > 3		
$10 == "10"$		
$10 === "10"$		
NOT (true AND false)		
$(5 + 5) \times (2 + 3)$		

Exercise 1.4: Spot the Type Error

Instructions: Each expression below has an error. Identify the error type and explain why it's wrong.

Expression	Error?	Why is it wrong?
"Hello" * 3		
5 + true		

Expression	Error?	Why is it wrong?
[1,2] + 3		
10 / 0		
"5" + 3		

Exercise 1.5: Complete the Data Map

Instructions: You're building a student management system. Identify the data types, variables, and operations you would use.

Scenario:

- Students have names, ages, and grades
- Grades are a list of numbers
- Each student has a passing status (true/false)
- Sometimes students have missing grades (null)

Data Type	Variables	Operations (2-3 each)
Number		
Text		
Boolean		
List		
Nothing		

Day 1 Summary

Key Takeaways

1. **Data Types:** Number, Text, Boolean, List, Nothing
2. **Variables:** Names that point to values. In FP, they never change (immutable).
3. **Operators:** Actions you can perform on data (add, compare, combine, etc.)
4. **Type Errors:** Happen when you mix wrong types

Quiz Yourself

1. What are the 5 data types?

2. What is a variable?
3. What does "immutable" mean?
4. What operator do you use to combine text?
5. What is a type error?

Day 1 Homework

Think about a process you do every day (making coffee, commuting, cooking dinner). Write down:

1. What data types are involved?
2. What variables would you name?
3. What operations do you perform?

Day 2: Expressions, Statements & Conditions

Learning Objectives

By the end of Day 2, you will be able to:

- Distinguish between expressions and statements
- Write expressions that produce values
- Identify statements that perform actions
- Build decision trees using conditions
- Create IF/THEN/ELSE logic

2.1 Expressions

What Is an Expression?

An **expression** is a question that produces a value.

Examples:

What **is** `2 + 2`? → `4`
What **is** `"Hello" + " World"`? → `"Hello World"`
Is `10 > 5`? → `true`
What **is** the first item **in** `[1,2,3]`? → `1`

Key point: Every expression gives you something back. You can use the result immediately.

Characteristics of Expressions

- **Always produce a value**
- **No side effects** (nothing else happens)
- **Predictable** (same input = same output)

Real-world analogy: Asking a question: "What is the capital of France?" → Answer: "Paris". Nothing else happens. That's an expression.

Examples of Expressions

Expression	Produces
<code>5 + 3</code>	8
<code>"Hello" + "!"</code>	<code>"Hello!"</code>
<code>age >= 18</code>	true or false
<code>scores[0]</code>	First score
<code>name.length</code>	Number of letters

Expressions vs. Statements

Expression	Statement
Asks a question	Gives a command
Produces a value	Performs an action
No side effects	Has side effects
Example: <code>5 + 3</code>	Example: <code>Print "Hello"</code>

2.2 Statements

What Is a Statement?

A **statement** is a command that performs an action. It does not produce a value.

Examples:

```
Print "Hello" to the screen
Save this file
Send an email
Turn on the light
```

Key point: Statements do things. They don't give you a value you can use.

Characteristics of Statements

- **No return value** (or returns nothing useful)
- **Have side effects** (change the world outside)
- **Less predictable** (depend on external state)

Real-world analogy: Giving a command: "Walk to the door and open it." → Something happens (door opens), but nothing is returned to you. That's a statement.

Examples of Statements

Statement	What it does
Print "Hello"	Shows "Hello" on screen
Save file	Writes file to disk
Send email	Sends an email
Update balance	Changes bank balance

The Functional Programming Rule

All logic should be expressions. Only input and output are statements.

Good:

```
total = price * quantity
discount = total * 0.1
finalPrice = total - discount
Print finalPrice ← Only statement at the end
```

Bad (too many statements):

```
Set total = 0
Add price to total
Calculate discount
```

```
Update total = total - discount
```

```
Print total ← Should be only the final step
```

2.3 Conditions

What Is a Condition?

A **condition** is a decision point in your program. It allows the program to choose between different paths based on data.

Structure:

```
IF (condition is true) THEN do this  
ELSE do something else
```

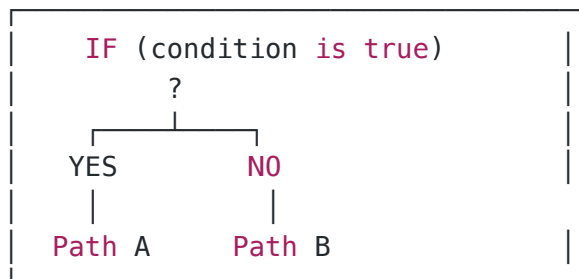
Example:

```
IF (age >= 18) THEN print "Adult"  
ELSE print "Child"
```

How Conditions Work

1. The condition must be a boolean expression (true or false)
2. If true, the program takes one path
3. If false, the program takes another path
4. You can have multiple conditions in a chain

Flow Chart of a Condition



Examples of Conditions

Simple Condition:

```
IF (score >= 60) THEN print "Pass"  
ELSE print "Fail"
```

Multiple Conditions:

```
IF (score > 90) THEN print "A"  
ELSE IF (score > 80) THEN print "B"  
ELSE IF (score > 70) THEN print "C"  
ELSE IF (score > 60) THEN print "D"  
ELSE print "F"
```

Complex Condition with AND:

```
IF (isStudent AND hasID) THEN print "Student discount"  
ELSE print "Regular price"
```

Complex Condition with OR:

```
IF (isWeekend OR isHoliday) THEN print "Special hours"  
ELSE print "Regular hours"
```

2.4 Decision Trees

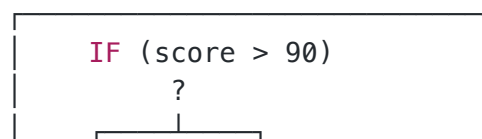
What Is a Decision Tree?

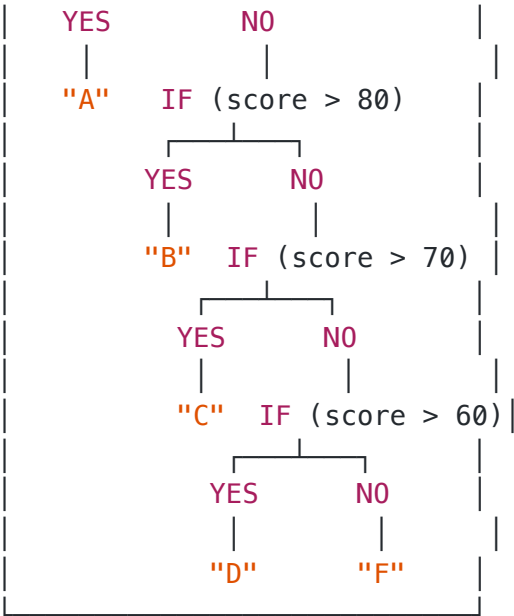
A **decision tree** is a visual representation of conditions. It shows all possible paths through your logic.

How to Draw a Decision Tree

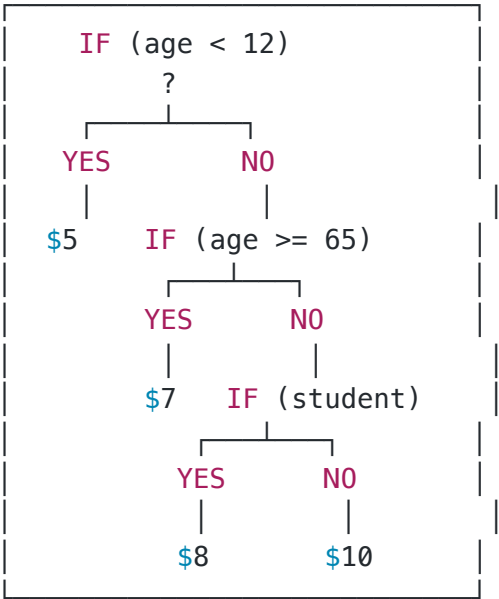
1. Start with the first condition at the top
2. Branch into YES and NO paths
3. For each branch, add the next condition if needed
4. Continue until you reach the final outcome

Example Decision Tree: Grading System





Example Decision Tree: Ticket Pricing



Day 2 Exercises

Exercise 2.1: Expression or Statement?

Instructions: For each item, write "E" if it's an expression (produces a value) or "S" if it's a statement (performs an action).

Item	E or S?	Why?
Add 5 and 3		

Item	E or S?	Why?
Print "Hello"		
What is the capital of France?		
Write your name on the board		
Calculate the area of a circle		
Save this file		
Is 10 greater than 5?		
Send an email		
Find the length of "Hello"		
Turn on the light		

Exercise 2.2: Spot the Statement

Instructions: Below is a recipe with statements and expressions mixed. Circle all statements and underline all expressions.

Recipe: Calculate total price with discount

1. Set `total` = 0
2. For each item `in` cart:
3. Add `item.price` to `total`
4. Calculate `discount` = `total` × 0.1
5. Update `total` = `total` - `discount`
6. Print `total`

Statements (circle): _____

Expressions (underline): _____

Rewritten (FP Style): Write the recipe so only the final line is a statement.

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

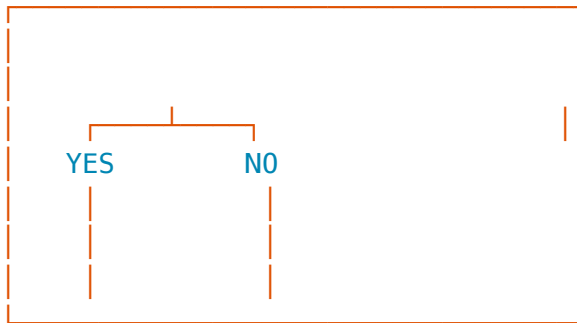
Exercise 2.3: Build a Decision Tree

Instructions: Draw a decision tree for each scenario.

Scenario 1: Voting Eligibility

A program that checks if someone can vote. If age ≥ 18 , print "Eligible". Otherwise, print "Not eligible".

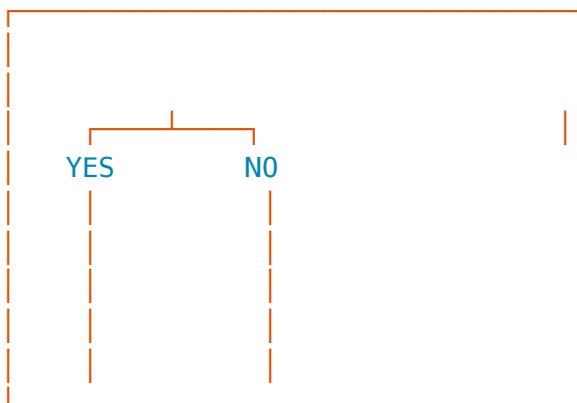
Draw your decision tree here:



Scenario 2: Grading System

A grading system. If score > 90 , grade = A. If score > 80 , grade = B. If score > 70 , grade = C. Otherwise, grade = F.

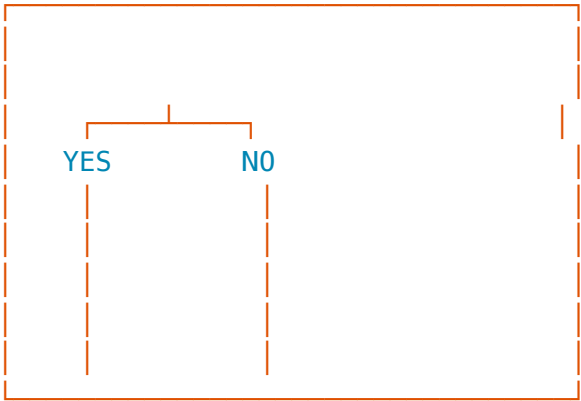
Draw your decision tree here:



Scenario 3: Weather App

A weather app. If temperature > 30 , print "Hot". Else if temperature > 20 , print "Warm". Else if temperature > 10 , print "Cool". Else print "Cold".

Draw your decision tree here:



Exercise 2.4: Complete the Decision System

Instructions: Design the complete decision system for the cinema ticket pricing scenario below.

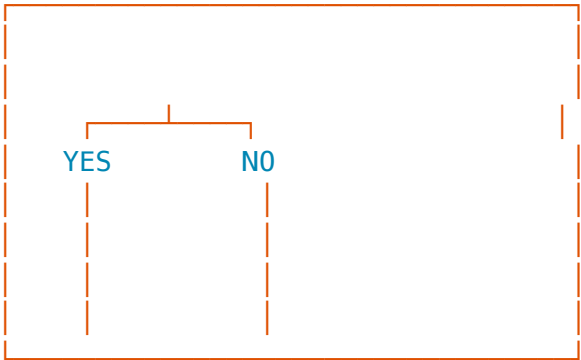
Scenario:

- Regular ticket: \$10
- Student ticket: \$8 (must be 18+ AND have student ID)
- Child ticket: \$5 (must be under 12)
- Senior ticket: \$7 (must be 65+)

Variables Needed:

Variable Name	Data Type

Decision Tree:



Final Price Calculation (Expressions):

- 1.
- 2.
- 3.

Statement (Only one at the end):

- 1.

Day 2 Summary

Key Takeaways

1. **Expressions:** Questions that produce values. Use for ALL logic.
2. **Statements:** Commands that perform actions. Use ONLY for input/output.
3. **Conditions:** IF/THEN/ELSE decisions based on booleans.
4. **Decision Trees:** Visual representation of conditions.
5. **FP Rule:** Push statements to the edges. Use expressions everywhere else.

Quiz Yourself

1. What is an expression?
2. What is a statement?
3. What is the FP rule for expressions vs statements?
4. What is a condition?
5. How do you draw a decision tree?

Day 2 Homework

Think about a decision you make daily (what to wear, what to eat for lunch, etc.). Draw a decision tree showing all the possible choices.

Day 3: Loops, Lists & Data Structures

Learning Objectives

By the end of Day 3, you will be able to:

- Use loops to repeat actions

- Work with lists as ordered collections
- Create records with named fields
- Combine lists and records into data structures
- Apply loops to real-world scenarios

3.1 Loops

What Is a Loop?

A **loop** is a way to repeat instructions multiple times without writing them out.

The Problem Without Loops:

```
Print "Hello!"  
Print "Hello!"  
Print "Hello!"  
Print "Hello!"  
Print "Hello!"
```

(5 lines of duplicate code)

The Solution With Loops:

```
FOR each item in [1,2,3,4,5]:  
    Print "Hello!"
```

(2 lines, does the same thing)

Three Types of Loops

1. FOR EACH Loop

Used when you have a list and you want to do something for every item.

Structure:

```
FOR each item in list:  
    do something with item
```

Examples:

```
FOR each name in ["Alice", "Bob", "Charlie"]:  
    Print "Hello, " + name  
  
FOR each number in [1,2,3,4,5]:  
    Print number × 2  
  
FOR each score in [85, 90, 78, 92]:  
    total = total + score
```

Real-world analogy: Washing dishes: "For each plate in the stack, wash it, dry it, put it away."

2. WHILE Loop

Used when you want to repeat until a condition changes.

Structure:

```
WHILE condition is true:  
    do something  
    (something must change the condition)
```

Examples:

```
WHILE (guess != password):  
    Ask for password  
    Get guess  
  
WHILE (balance < target):  
    Add interest to balance  
    count = count + 1
```

Real-world analogy: At a security gate: "While the password is wrong, keep asking for the password."

3. REPEAT N Times

Used when you know exactly how many times to repeat.

Structure:

```
REPEAT N times:  
    do something
```

Examples:

```
REPEAT 5 times:  
  Print "Hello!"
```

```
REPEAT 3 times:  
  Ask for input
```

Real-world analogy: A warm-up exercise: "Do 10 jumping jacks."

Loops Summary Table

Loop Type	When to Use	Structure
FOR EACH	You have a list to process	FOR each item in list:
WHILE	Repeat until condition changes	WHILE (condition):
REPEAT N	You know exactly how many times	REPEAT N times:

3.2 Lists

What Is a List?

A **list** is an ordered collection of items. Items are in a specific order, and you can access them by position.

Examples:

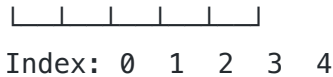
```
[10, 20, 30, 40, 50]  
["apple", "banana", "cherry"]  
[true, false, true, false]  
["Alice", 25, true]    (mixed types – usually avoid this)  
[]                     (empty list)
```

List Positions (Index)

Lists start counting at position 0 (zero-based indexing).

List: [10, 20, 30, 40, 50]

10	20	30	40	50
----	----	----	----	----



Accessing Items:

- First item: `list[0]` → 10
- Second item: `list[1]` → 20
- Last item: `list[4]` → 50
- Or: `list[length - 1]` → 50

List Operations

Operation	How To	Example	Result
Count items	<code>.length</code>	<code>[1,2,3].length</code>	3
First item	<code>[0]</code>	<code>[1,2,3][0]</code>	1
Last item	<code>[length-1]</code>	<code>[1,2,3][2]</code>	3
Combine	<code>+</code>	<code>[1,2] + [3,4]</code>	<code>[1,2,3,4]</code>
Check if contains	<code>item IN list</code>	<code>2 IN [1,2,3]</code>	true

3.3 Records/Objects

What Is a Record?

A **record** (also called an object) is a collection of named fields. Each field has a name and a value.

Example:

```
person = {
  name: "Alice",
  age: 25,
  city: "London",
  isStudent: true,
  scores: [85, 90, 78]
}
```

Accessing Record Fields

Operation	How To	Example	Result
Get value	<code>.fieldName</code>	<code>person.name</code>	"Alice"
Get value	<code>["fieldName"]</code>	<code>person["age"]</code>	25
Get value	<code>.fieldName</code>	<code>person.scores</code>	[85,90,78]

Records vs. Lists

List	Record
Ordered by position	Named fields
Items are numbered (0,1,2)	Items have labels (name, age)
All items usually same type	Different types allowed
Example: [10,20,30]	Example: {name:"Alice", age:25}

3.4 Data Structures

What Are Data Structures?

Data structures are ways to organize data. You can combine lists and records to create complex structures.

Example: List of Records

```
students = [  
  { name: "Alice", age: 20, grades: [85, 90, 78] },  
  { name: "Bob", age: 22, grades: [55, 60, 65] },  
  { name: "Charlie", age: 19, grades: [92, 88, 94] }  
]
```

Accessing Nested Data

```
students[0].name → "Alice"  
students[1].grades → [55, 60, 65]  
students[2].grades[0] → 92
```

Example: Record with Nested List

```
order = {
  id: 1001,
  customer: "John Doe",
  items: [
    { name: "Book", price: 20, quantity: 2 },
    { name: "Pen", price: 5, quantity: 3 },
    { name: "Notebook", price: 10, quantity: 1 }
  ],
  total: 65
}
```

Common Data Structures

Structure	Example	Use Case
List of numbers	[10,20,30]	Scores, temperatures
List of text	["a","b","c"]	Names, items
Record	{name:"Alice", age:25}	Person, product
List of records	[{name:"Alice"},{name:"Bob"}]	Multiple people, products
Record with list	{name:"Alice", scores:[85,90]}	Person with multiple scores

Day 3 Exercises

Exercise 3.1: Manual Loop Simulation

Instructions: Simulate a FOR EACH loop for the following lists.

List 1: Names

```
names = ["Alice", "Bob", "Charlie", "David"]
```

Loop instruction: For each name, print "Hello, [name]!"

Show the output:

```
Hello, _____
Hello, _____
Hello, _____
Hello, _____
```

List 2: Numbers

```
numbers = [10, 20, 30, 40, 50]
```

Loop instruction: For each number, double it.

Show your work:

```
Loop 1: _____ → _____  
Loop 2: _____ → _____  
Loop 3: _____ → _____  
Loop 4: _____ → _____  
Loop 5: _____ → _____
```

Exercise 3.2: While Loop Simulation

Instructions: Simulate a WHILE loop.

Scenario: Keep asking for a password until the user types "1234".

Show the simulation:

```
Guess 1: _____ → _____  
Guess 2: _____ → _____  
Guess 3: _____ → _____
```

When did the loop stop? _____

Exercise 3.3: Shopping Cart

Instructions: You have a shopping cart with the following items. Calculate the total price.

Cart:

```
items = [  
  { name: "Apple", price: 1.00 },  
  { name: "Banana", price: 2.00 },  
  { name: "Cherry", price: 3.00 },  
  { name: "Date", price: 4.00 },  
  { name: "Elderberry", price: 5.00 }  
]
```

Loop algorithm:

1. total = 0
2. FOR each item in items:
3. total = total + item.price
4. Output total

Show your work:

Loop 1: total = 0 + 1.00 = _____
Loop 2: total = _____ + 2.00 = _____
Loop 3: total = _____ + 3.00 = _____
Loop 4: total = _____ + 4.00 = _____
Loop 5: total = _____ + 5.00 = _____

Final Total: \$ _____

Exercise 3.4: Student Record

Instructions: Design a student record and calculate the average grade.

Student Data:

```
student = {  
  name: "Alice Johnson",  
  age: 20,  
  city: "London",  
  subjects: ["Math", "Science", "History", "English"],  
  grades: [85, 92, 78, 88]  
}
```

Calculate the average:

1. total = _____
2. FOR each grade in grades:
3. total = total + _____
4. average = total / _____
5. Output average

Show your work:

Grades: [85, 92, 78, 88]
Loop 1: total = 0 + 85 = _____
Loop 2: total = _____ + 92 = _____
Loop 3: total = _____ + 78 = _____
Loop 4: total = _____ + 88 = _____

Average = _____ / 4 = _____

Exercise 3.5: Complete Data System

Instructions: Design a complete class management system.

Data:

```
students = [  
  { name: "Alice", grades: [85, 90, 78] },  
  { name: "Bob", grades: [55, 60, 65] },  
  { name: "Charlie", grades: [92, 88, 94] }  
]
```

Tasks:

1. Calculate the average for each student
2. Determine if each student passes (average ≥ 60)
3. Count how many students pass
4. Calculate the class average

Show your work:

Student	Grades	Average	Pass (≥ 60)?
Alice	[85,90,78]		
Bob	[55,60,65]		
Charlie	[92,88,94]		

Number of passing students: _____

Class average: _____ (sum of all averages / number of students)

Day 3 Summary

Key Takeaways

1. **Loops:** Repeat actions without writing duplicate code
2. **Three loops:** FOR EACH (lists), WHILE (conditions), REPEAT N (count)
3. **Lists:** Ordered collections with positions (index 0,1,2,...)

4. **Records:** Named fields (like a filing cabinet with labeled drawers)
5. **Data Structures:** Combine lists and records for complex data

Quiz Yourself

1. What are the three types of loops?
2. What is the difference between a list and a record?
3. What is zero-based indexing?
4. How do you access the first item in a list?
5. How do you access a field in a record?

Day 3 Homework

Think about your favorite app (Instagram, Spotify, etc.). What lists and records do you think it uses? Write down at least 3 data structures.

Day 4: Functions, Scope & Errors

Learning Objectives

By the end of Day 4, you will be able to:

- Create functions to package logic
- Use parameters to pass inputs to functions
- Return values from functions
- Understand variable scope (local vs global)
- Identify and fix three types of errors

4.1 Functions

What Is a Function?

A **function** is a named block of logic that you can reuse. Like a recipe, you write it once and use it many times.

Analogy: A chocolate cake recipe. You don't re-invent the recipe every time. You just say "follow the cake recipe."

Parts of a Function

```
FUNCTION name(parameters):  
    processing logic  
    return result
```

Parts:

1. **NAME:** What do you call it?
2. **PARAMETERS:** What inputs does it need?
3. **PROCESSING:** What does it do?
4. **RETURN:** What does it give back?

Simple Function Examples

Function 1: Double a Number

```
FUNCTION double(number):  
    return number * 2
```

```
double(5) → 10  
double(10) → 20  
double(0.5) → 1
```

Function 2: Greet Someone

```
FUNCTION greet(name):  
    return "Hello, " + name
```

```
greet("Alice") → "Hello, Alice"  
greet("Bob") → "Hello, Bob"
```

Function 3: Check if Adult

```
FUNCTION isAdult(age):  
    return age >= 18
```

```
isAdult(25) → true  
isAdult(16) → false
```

Function 4: Calculate Circle Area

```
FUNCTION circleArea(radius):  
    return 3.14159 * radius * radius
```

```
circleArea(5) → 78.54  
circleArea(10) → 314.16
```

Function 5: Find Maximum

```
FUNCTION max(a, b):  
  IF a > b THEN return a  
  ELSE return b
```

```
max(10, 20) → 20  
max(5, 3) → 5
```

Functions Can Call Other Functions

```
FUNCTION rectangleArea(width, height):  
  return width * height
```

```
FUNCTION rectangleVolume(width, height, depth):  
  area = rectangleArea(width, height)  
  return area * depth
```

```
rectangleVolume(2, 3, 4) → 24
```

4.2 Scope

What Is Scope?

Scope is the visibility of a variable. Where can you see it? Where does it exist?

Two Types of Scope

1. Local Scope

Variables declared inside a function are **local** to that function. They can only be seen inside that function.

```
FUNCTION greet():  
  LOCAL: greeting = "Hello"  
  PRINT greeting    ✓ Works
```

```
FUNCTION otherFunction():  
  PRINT greeting    ✗ ERROR (greeting doesn't exist here)
```

Analogy: Your bedroom. Only you can see what's inside.

2. Global Scope

Variables declared outside all functions are **global**. They can be seen everywhere.

```
GLOBAL: appName = "My App"
```

```
FUNCTION greet():  
    PRINT appName    ✓ Works (global)
```

```
FUNCTION otherFunction():  
    PRINT appName    ✓ Works (global)
```

Analogy: The kitchen. Everyone can see what's in the kitchen.

Scope Rules

Scope	Where Declared	Visible Where
Local	Inside a function	Only inside that function
Global	Outside all functions	Everywhere
Parameter	In function definition	Only inside that function

Scope Example

```
GLOBAL: city = "London"
```

```
FUNCTION getWeather():  
    LOCAL: temperature = 25  
    PRINT temperature    ✓ (local)  
    PRINT city            ✓ (global)
```

```
FUNCTION printGreeting():  
    LOCAL: greeting = "Hello"  
    PRINT greeting        ✓ (local)  
    PRINT temperature     x ERROR (local to getWeather)  
    PRINT city             ✓ (global)
```

Best Practice

Keep variables as local as possible. Use global variables sparingly (only for constants).

```
GOOD (local): FUNCTION calculate(): price = 10  
BETTER (local): FUNCTION calculate(): price = 10 (only used here)
```

AVOID (global): GLOBAL price = 10 (used everywhere, risk of conflicts)

4.3 Errors

What Are Errors?

Errors are problems in your program. They happen when something goes wrong.

Three Types of Errors

1. Syntax Error

You wrote something that doesn't make sense (like a grammar mistake).

Example:

```
5 + * 3    ← What is * doing here?
```

Analogy: "I cake eat" instead of "I eat cake." The sentence doesn't make sense.

Common syntax errors:

- Missing parentheses
- Extra operators
- Misspelled keywords
- Unclosed quotes

2. Runtime Error

Your code is syntactically correct, but something breaks when it runs.

Example:

```
10 / 0    ← Can't divide by zero
```

Analogy: "Add salt" but you have no salt. The instruction is valid, but it fails when executed.

Common runtime errors:

- Division by zero
- Accessing a list item that doesn't exist

- Using a variable that hasn't been defined
- Opening a file that doesn't exist

3. Logic Error

Your code runs without errors, but gives the WRONG answer.

Example:

```
FUNCTION rectangleArea(width, height):  
    return width + height    ← WRONG! Should be width × height
```

Analogy: "Bake at 350°F" but you use 180°F. The instructions are valid, but the result is wrong.

Common logic errors:

- Using + instead of *
- Using > instead of <
- Off-by-one errors (starting at 1 instead of 0)
- Forgetting to update a loop variable

Error Comparison Table

Error Type	Runs?	Gives Answer?	Hard to Find?
Syntax	No	No	Easy
Runtime	Yes (crashes)	No	Medium
Logic	Yes	Wrong answer	Hardest

Day 4 Exercises

Exercise 4.1: Write Recipes (Functions)

Instructions: Write a function ("recipe") for each of the following.

1. Square a Number

```
FUNCTION square(____):  
    return _____
```

2. Check if Adult

```
FUNCTION isAdult(____):  
    return _____
```

3. Find the Larger Number

```
FUNCTION max(____, ____):  
    IF ____ THEN return ____  
    ELSE return ____
```

4. Calculate Text Length

```
FUNCTION textLength(____):  
    return _____
```

5. Sum All Numbers in a List

```
FUNCTION sumAll(____):  
    total = 0  
    FOR each ____:  
        total = total + ____  
    return _____
```

Exercise 4.2: Function Assembly Line

Instructions: You have three functions. Chain them together.

Functions:

1. `capitalize(word)`: returns word with first letter capitalized
"hello" → "Hello"
2. `addExclamation(word)`: returns word with "!" added
"Hello" → "Hello!"
3. `reverse(word)`: returns word reversed
"Hello!" → "!olleH"

Task: Chain all three functions together starting with "hello".

Step 1: `capitalize("hello")` → _____

Step 2: addExclamation(_____) → _____

Step 3: reverse(_____) → _____

Final Output: _____

Write the full chain:

reverse(addExclamation(capitalize("hello"))) → _____

Exercise 4.3: Scope Game

Instructions: For each scenario, identify which variables are visible.

Scenario 1:

GLOBAL: city = "London"

```
FUNCTION getWeather():  
  LOCAL: temperature = 25  
  PRINT temperature  
  PRINT city
```

What can getWeather() see?

- temperature: YES / NO (circle one)
- city: YES / NO (circle one)

Scenario 2:

```
FUNCTION printName():  
  LOCAL: name = "Alice"  
  PRINT name
```

```
FUNCTION printGreeting():  
  LOCAL: greeting = "Hello"  
  PRINT greeting  
  PRINT name
```

What can printGreeting() see?

- greeting: YES / NO (circle one)
- name: YES / NO (circle one)

Scenario 3:

```
GLOBAL: appName = "MyApp"
```

```
FUNCTION funcA():  
  LOCAL: x = 5  
  PRINT x  
  PRINT appName
```

```
FUNCTION funcB():  
  LOCAL: y = 10  
  PRINT y  
  PRINT x  
  PRINT appName
```

What can funcB() see?

- y: YES / NO (circle one)
- x: YES / NO (circle one)
- appName: YES / NO (circle one)

Exercise 4.4: Fix the Broken Recipes

Instructions: Each recipe has an error. Identify the type and fix it.

Error 1:

```
FUNCTION double(number)  
  return number * 2
```

Error Type: Syntax / Runtime / Logic (circle one) **Fix:** _____

Error 2:

```
FUNCTION average(scores):  
  total = 0  
  FOR each score in scores:  
    total = total + score  
  return total / 0
```

Error Type: Syntax / Runtime / Logic (circle one) **Fix:** _____

Error 3:

```
FUNCTION discountPrice(price):  
    return price - 10
```

(Should be 10% off, not \$10 off) Error Type: Syntax / Runtime / Logic (circle one) Fix:

Error 4:

```
FUNCTION greet(name):  
    return "Hello, " + name  
  
greet("Alice") → "Hello, Alice" (this works)
```

Error Type: Syntax / Runtime / Logic (circle one) Fix: Nothing! This is correct. Only mark if you find an error.

Exercise 4.5: Complete Function System

Instructions: Design a complete function system for an e-commerce checkout.

Scenario:

- Calculate subtotal (sum of all item prices)
- Apply discount (10% off for coupon "SAVE10")
- Add tax (10% tax)
- Return final total

Write the functions:

Function 1: calculateSubtotal

```
FUNCTION calculateSubtotal(cart):  
    total = 0  
    FOR each _____:  
        total = total + _____  
    return _____
```

Function 2: applyDiscount

```
FUNCTION applyDiscount(subtotal, coupon):  
    IF coupon == "SAVE10":  
        return _____
```

ELSE:

return _____

Function 3: addTax

FUNCTION addTax(total, taxRate):

return _____

Function 4: calculateTotal

```
FUNCTION calculateTotal(cart, coupon, taxRate):  
  subtotal = calculateSubtotal(_____  
  discounted = applyDiscount(_____, _____)  
  final = addTax(_____, _____)  
  return _____
```

Test with:

```
cart = [{name: "Book", price: 20}, {name: "Pen", price: 5}]  
coupon = "SAVE10"  
taxRate = 0.1
```

calculateTotal(cart, coupon, taxRate) → ?

Show your work:

```
subtotal = ____ + ____ = ____  
discounted = ____ × 0.9 = ____  
tax = ____ × 0.1 = ____  
final = ____ + ____ = ____
```

Final Total: \$ ____

Day 4 Summary

Key Takeaways

1. **Functions:** Named blocks of logic. Write once, use everywhere.
2. **Parts:** Name, Parameters, Processing, Return.
3. **Scope:** Local (inside functions) vs Global (everywhere).
4. **FP Rule:** Keep variables local as much as possible.

5. **Three errors:** Syntax (grammar), Runtime (breaks when running), Logic (wrong answer).

Quiz Yourself

1. What are the four parts of a function?
2. What is local scope?
3. What is global scope?
4. What are the three types of errors?
5. Which error type is the hardest to find?

Day 4 Homework

Design a function for each:

1. "Calculate the area of a triangle"
2. "Check if a number is prime"
3. "Find the largest number in a list"

Day 5: Input/Output, Algorithms & Complete Design

Learning Objectives

By the end of Day 5, you will be able to:

- Handle input from users and files
- Produce output to screens and files
- Design algorithms for real-world problems
- Build a complete program from scratch
- Apply all concepts to a final project

5.1 Input & Output

What Is Input & Output?

Input is data coming into your program. **Output** is data leaving your program.

INPUT → PROGRAM → OUTPUT

Sources of Input

Source	Examples
User typing	Keyboard input
Mouse clicks	Button presses
Files	Reading from disk
Internet	API calls
Sensors	Temperature, GPS

Destinations for Output

Destination	Examples
Screen	Text, graphics
Files	Saving to disk
Internet	Sending data
Printer	Physical copies
Speakers	Sound

The FP Rule

Keep input and output at the edges. Put all logic in the middle.

Good:

```
INPUT: Get user's age
↓
LOGIC: Determine if adult (age >= 18)
↓
OUTPUT: Show "Adult" or "Child"
```

Bad (logic in input/output):

```
INPUT: Get age AND determine if adult ← Too much in one step
OUTPUT: Show result AND save to file ← Too much in one step
```

5.2 Algorithms

What Is an Algorithm?

An **algorithm** is a step-by-step sequence of instructions to solve a problem.

Analogy: A recipe is an algorithm for cooking.

Three Classic Algorithms

1. Find the Maximum

Problem: Find the largest number in a list.

Algorithm:

1. Look at the first number → **call** it **"max"**
2. Look **at** the **next** number
3. **If** this number **is** larger **than** **"max"**:
4. **Update** **"max"** to this number
5. **Repeat** steps 2–4 **until** **no** more numbers
6. **Output** **"max"**

Example:

List: [3, 7, 2, 9, 1]

Step 1: max = 3

Step 2–4: Compare 3 **and** 7 → 7 **is** larger → max = 7

Step 2–4: Compare 7 **and** 2 → 7 **is** larger → max = 7

Step 2–4: Compare 7 **and** 9 → 9 **is** larger → max = 9

Step 2–4: Compare 9 **and** 1 → 9 **is** larger → max = 9

Output: 9

2. Search for an Item

Problem: Check if a specific item exists in a list.

Algorithm:

1. Look **at** the **first** **item**
2. If **it** matches **the** search **item** → **return** **true**
3. If **not**, look **at** the next **item**

4. Repeat steps 1–3 **until** no **items** left
5. If no match found → **return false**

Example:

List: ["apple", "banana", "cherry"]

Search for: "banana"

Step 1: Check "apple" → no match

Step 2: Check "banana" → match → return true

3. Sort a List

Problem: Arrange a list from smallest to largest.

Algorithm (Simple - Bubble Sort):

1. Compare the first two items
2. If the first is larger than the second, swap them
3. Move to the next pair (second and third)
4. Repeat steps 1–3 until the end of the list
5. If any swaps were made, go back to step 1
6. If no swaps were made, the list is sorted

Example:

List: [5, 2, 8, 1, 3]

Pass 1:

[5, 2, 8, 1, 3]

Compare 5 & 2 → swap → [2, 5, 8, 1, 3]

Compare 5 & 8 → ok → [2, 5, 8, 1, 3]

Compare 8 & 1 → swap → [2, 5, 1, 8, 3]

Compare 8 & 3 → swap → [2, 5, 1, 3, 8]

Pass 2:

[2, 5, 1, 3, 8]

Compare 2 & 5 → ok

Compare 5 & 1 → swap → [2, 1, 5, 3, 8]

Compare 5 & 3 → swap → [2, 1, 3, 5, 8]

Pass 3:

[2, 1, 3, 5, 8]

Compare 2 & 1 → swap → [1, 2, 3, 5, 8]

Compare 2 & 3 → ok

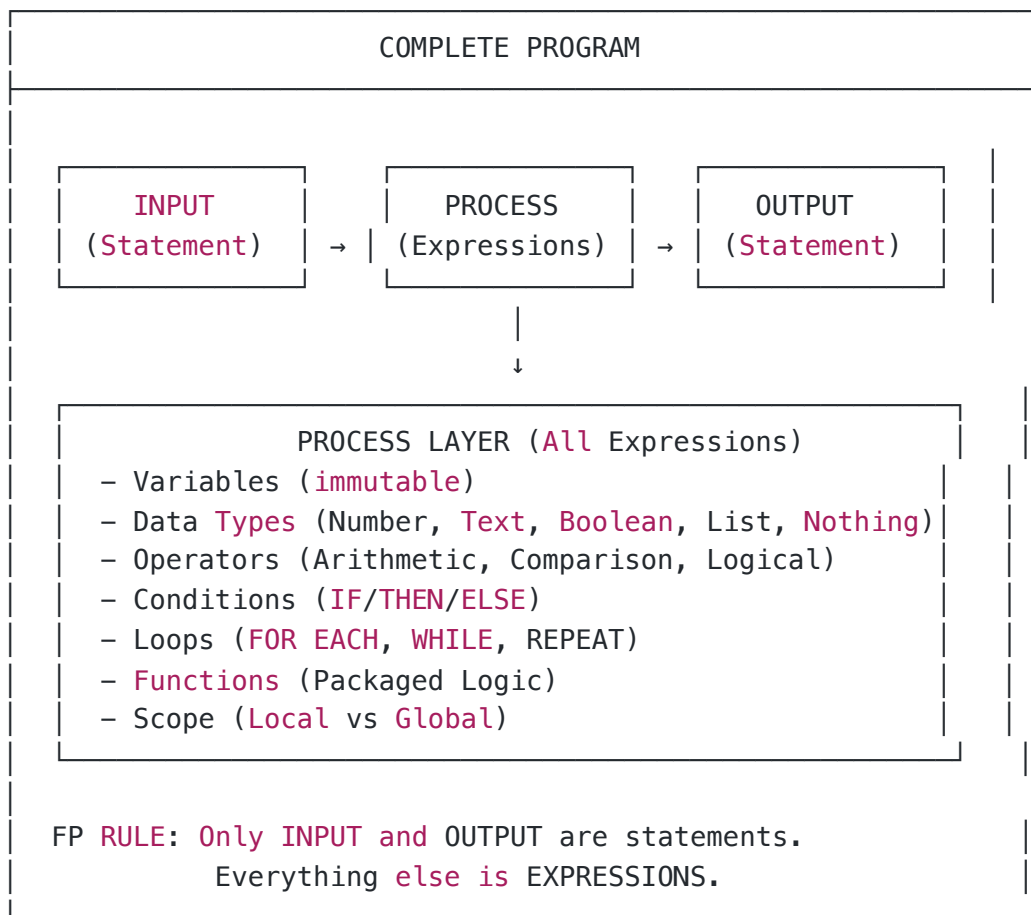
Compare 3 & 5 → ok

Sorted: [1, 2, 3, 5, 8]

5.3 Complete Program Structure

The Full Program

A complete program follows this structure:



Example: Complete Program

Program: Student Grade Calculator

INPUT:

```

students = [
  { name: "Alice", scores: [85, 90, 78] },
  { name: "Bob", scores: [55, 60, 65] },
  { name: "Charlie", scores: [92, 88, 94] }
]
  
```

PROCESS (All expressions):

```
FOR each student:  
    total = sum of scores  
    average = total / length of scores  
    IF average >= 60 THEN status = "Pass" ELSE status = "Fail"
```

OUTPUT:

```
Print "Alice: 84.33 - Pass"  
Print "Bob: 60.00 - Pass"  
Print "Charlie: 91.33 - Pass"  
Print "Class Average: 78.55"
```

Day 5 Exercises

Exercise 5.1: Algorithm Steps

Instructions: Write the algorithm for each problem.

Problem 1: Find the Youngest Person

Algorithm: Find Youngest

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Problem 2: Check if a List Contains the Number 10

Algorithm: Find Number 10

- 1.
- 2.
- 3.
- 4.
- 5.

Problem 3: Calculate the Average of a List

Algorithm: Calculate Average

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Exercise 5.2: Waiter Simulation (I/O)

Instructions: Act out the customer-waiter-kitchen flow.

Customer (Input): "I want a large latte with extra shot."

Waiter (Process): Write down the order on an index card.

Kitchen (Output): Write the receipt.

Receipt:

Order: Large Latte + Extra Shot

What did the Waiter change? _____

What was the output? _____

Exercise 5.3: Complete Program Design

Instructions: Design a complete program for the student grade calculator scenario.

Scenario:

A program that takes a list of student scores, calculates the average, and prints "Pass" if average ≥ 60 , otherwise "Fail".

1. Data Types Used:

Number: _____

Text: _____

Boolean: _____

List: _____

Nothing: _____

2. Variables Needed:

3. Expressions:

4. Conditions:

5. Loops:

6. Functions:

FUNCTION calculateAverage(scores):

FUNCTION determineResult(average):

IF _____
ELSE _____

7. Scope:

Local: _____
Global: _____

8. Input:

9. Output:

10. Algorithm (Step-by-Step):

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.

Exercise 5.4: Final Project - Pizza Shop

Instructions: Design a complete program for the pizza shop.

Scenario:

You run a pizza shop. Customers can order custom pizzas.

PRICING:

- Base **pizza**: \$10
- Each **topping**: +\$2
- Large **size**: +\$5
- Family **size**: +\$10
- **Discount**: 10% **if** total > \$30
- **Tax**: 10% (applied after discount)

INPUT:

- **Size**: "small", "large", or "family"
- **Toppings**: list of toppings (e.g., ["pepperoni", "mushrooms", "olives"])
- **Coupon**: "SAVE10", "SAVE20", or none

OUTPUT:

- Order summary
- Subtotal
- Discount (**if** applied)
- Tax
- Final total

Your Design:

1. Data Types Needed:

2. Variables (Immutable):

3. Expressions:

4. Conditions:

5. Loops:

6. Functions:

FUNCTION calculateSubtotal(size, toppings):

FUNCTION getSizeCost(size):

IF _____
ELSE IF _____

ELSE _____

FUNCTION applyDiscount(total):

IF _____

ELSE _____

FUNCTION applyCoupon(total, coupon):

IF _____

ELSE IF _____

ELSE _____

FUNCTION calculateTotal(size, toppings, coupon):

7. Scope:

Global: _____

Local: _____

8. Input:

9. Output:

Order Summary

=====

Size: _____

Toppings: _____

Subtotal: \$ _____

Discount: -\$ _____

Tax: \$ _____

Total: \$ _____

10. Algorithm (Step-by-Step):

1.

2.

3.

4.

5.

6.

- 7.
- 8.
- 9.
- 10.

11. Error Handling:

Invalid size: _____
Empty toppings: _____
Invalid coupon: _____

Exercise 5.5: Test Your Program

Instructions: Test your pizza shop program with the following scenarios.

Test Case 1: Small Pizza, 2 Toppings, No Coupon

Input:
Size = "small"
Toppings = ["pepperoni", "mushrooms"]
Coupon = none

Calculate:
Subtotal = _____
Discount = _____
Final = _____
Tax = _____
Total = _____

Output:

Test Case 2: Large Pizza, 3 Toppings, SAVE10 Coupon

Input:
Size = "large"
Toppings = ["pepperoni", "mushrooms", "olives"]
Coupon = "SAVE10"

Calculate:
Subtotal = _____
Discount = _____
Final = _____
Tax = _____

Total = _____

Output:

Test Case 3: Family Pizza, 4 Toppings, Total > \$30

Input:

Size = "family"

Toppings = ["pepperoni", "mushrooms", "olives", "peppers"]

Coupon = none

Calculate:

Subtotal = _____

Discount = _____

Final = _____

Tax = _____

Total = _____

Output:

Day 5 Summary

Key Takeaways

1. **Input:** Data coming into the program (users, files, etc.)
2. **Output:** Data leaving the program (screen, files, etc.)
3. **Algorithms:** Step-by-step problem-solving
4. **Complete Program:** Input → Process (Expressions) → Output
5. **FP Rule:** Only input/output are statements. Everything else is expressions.

Quiz Yourself

1. What is input?
2. What is output?
3. What is an algorithm?
4. What is the FP rule for input/output?
5. What are the three classic algorithms?

Final Assessment

Comprehensive Program Design

Instructions: Design a complete program from scratch. Write everything in plain English. No code required.

Scenario:

"You are building a discount ticket booking system.

Rules:

- Regular tickets: \$50
- Student discount: 20% (must have student ID)
- Senior discount: 15% (must be 65+)
- Group discount: 10% for 5+ tickets
- VIP: Free upgrade for members

Input:

- Number of tickets
- Student status (yes/no)
- Senior status (yes/no)
- Member status (yes/no)

Output:

- Base price
- Discounts applied
- Final price
- VIP upgrade status"

Section 1: Data Types

List all data types used in this program:

Data Type	Example Value	Used For

Section 2: Variables

List all variables needed (immutable):

Variable Name	Value	Purpose

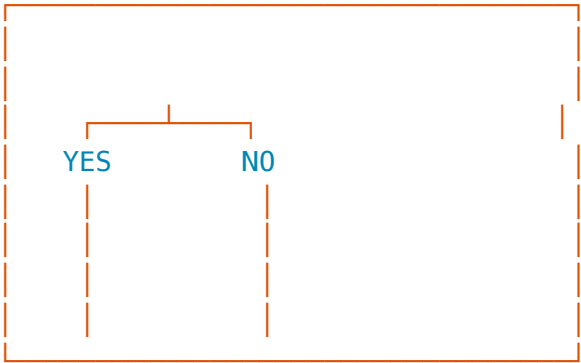
Section 3: Expressions

Write all expressions needed:

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Section 4: Conditions

Draw the decision tree:



Section 5: Loops

Is a loop needed? If yes, where and why?

Section 6: Functions

Design all functions:

FUNCTION calculateBasePrice(tickets):

FUNCTION applyStudentDiscount(price, isStudent):

FUNCTION applySeniorDiscount(price, isSenior):

FUNCTION applyGroupDiscount(price, tickets):

FUNCTION applyVIPUpgrade(isMember):

FUNCTION calculateFinal(tickets, isStudent, isSenior, isMember):

Section 7: Scope

Identify variable scope:

Variable	Scope (Local/Global)	Why?

Section 8: Input/Output

Input:

- 1.
- 2.
- 3.
- 4.

Output:

```
Order Summary
=====
Number of Tickets: ____
Base Price: $____
Student Discount: -$____
Senior Discount: -$____
Group Discount: -$____
VIP Upgrade: ____
Final Total: $____
=====
```

Section 9: Algorithm

Write the complete algorithm (step-by-step):

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.
- 11.
- 12.

Section 10: Error Handling

What errors could happen and how would you fix them?

Error	Fix

Section 11: Test Your Design

Test Case 1: 3 tickets, no discounts

Input:

```
tickets = 3  
isStudent = false  
isSenior = false  
isMember = false
```

Expected Output:

```
Base Price: $150  
Student Discount: $0  
Senior Discount: $0  
Group Discount: $0  
VIP Upgrade: No  
Final Price: $150
```

Your Calculation:

Test Case 2: 5 tickets, student, member

Input:

```
tickets = 5  
isStudent = true  
isSenior = false  
isMember = true
```

Expected Output:

```
Base Price: $250  
Student Discount: -$50  
Senior Discount: $0  
Group Discount: -$25  
VIP Upgrade: Yes  
Final Price: $175
```

Your Calculation:

Test Case 3: 6 tickets, senior, not member

Input:

```
tickets = 6  
isStudent = false
```

```
isSenior = true
isMember = false
```

Expected Output:
Base Price: \$300
Student Discount: \$0
Senior Discount: -\$45
Group Discount: -\$30
VIP Upgrade: No
Final Price: \$225

Your Calculation:

Final Assessment Scorecard

Section	Maximum Points	Your Score
1. Data Types	10	
2. Variables	10	
3. Expressions	10	
4. Conditions	10	
5. Loops	5	
6. Functions	15	
7. Scope	10	
8. Input/Output	10	
9. Algorithm	10	
10. Error Handling	5	
11. Test Cases	5	
Total	100	

Course Summary

All 13 Fundamentals

#	Concept	Day
1	Variables	1
2	Data Types	1
3	Operators	1
4	Expressions	2
5	Statements	2
6	Conditions	2
7	Loops	3
8	Lists	3
9	Records/Objects	3
10	Functions	4
11	Scope	4
12	Errors	4
13	Input/Output	5
14	Algorithms	5

The Golden Rules of Functional Programming

1. **Variables never change** (immutable)
2. **Use expressions for ALL logic** (produces values)
3. **Use statements ONLY for input/output** (performs actions)
4. **Keep functions pure** (no side effects)
5. **Push side effects to the edges** (input/output only)

Program Structure

INPUT (Statement)

↓

PROCESS (Expressions)

- Variables (immutable)
- Data Types
- Operators
- Conditions
- Loops
- Functions

— Scope



OUTPUT (Statement)

Key Insight

Programming is just: Data goes in → Functions transform it → Data goes out.

Glossary

A

Algorithm: A step-by-step sequence of instructions to solve a problem.

AND: A logical operator that returns true only if both conditions are true.

Array: Another name for a list. An ordered collection of items.

B

Boolean: A data type that is either true or false.

Branch: A path in a decision tree (YES or NO).

C

Comparison Operator: An operator that compares two values and returns true or false.

Condition: A decision point based on a boolean expression.

Constant: A variable that never changes (immutable).

D

Data Structure: A way to organize data (e.g., lists, records).

Data Type: A category of data (Number, Text, Boolean, List, Nothing).

Decision Tree: A visual representation of conditions.

E

ELSE: The path taken when a condition is false.

Error: A problem in a program (Syntax, Runtime, Logic).

Expression: A question that produces a value.

F

Field: A named value in a record.

FOR EACH Loop: A loop that processes every item in a list.

Function: A named block of reusable logic.

G

Global: Visible everywhere in the program.

I

IF: A condition that starts a decision.

Immutable: Cannot be changed once created.

Index: The position of an item in a list (starts at 0).

Input: Data coming into the program.

L

List: An ordered collection of items.

Local: Only visible inside a function.

Logic Error: A program that runs but gives the wrong answer.

Loop: A way to repeat instructions.

M

Maximum: The largest value in a list.

N

Nothing: A data type representing empty or missing values.

NOT: A logical operator that returns the opposite.

Number: A data type for numeric values.

O

Object: Another name for a record.

Operator: An action you can perform on data.

OR: A logical operator that returns true if at least one condition is true.

Output: Data leaving the program.

P

Parameter: An input to a function.

Pipeline: Connecting functions together (output of one becomes input of next).

Pure Function: A function with no side effects.

R

Record: A collection of named fields.

REPEAT N Loop: A loop that runs exactly N times.

Return: The output of a function.

Runtime Error: An error that occurs while the program is running.

S

Scope: The visibility of a variable.

Sort: To arrange a list in order.

Statement: A command that performs an action.

String: Another name for text data.

Syntax Error: An error in the grammar of the code.

T

Text: A data type for letters, words, and sentences.

True/False: Boolean values.

Type Error: An error from mixing wrong data types.

V

Variable: A name that points to a value.

W

WHILE Loop: A loop that repeats while a condition is true.

Z

Zero-Based Indexing: Lists start at position 0, not 1.

Course Completion Certificate

This certifies that

has successfully completed the

5-Day Functional Programming Foundations Course

covering all fundamental concepts including data types, variables, operators, expressions, statements, conditions, loops, lists, records, functions, scope, errors, input/output, and algorithms.

Date: _____

Instructor: _____

Signature: _____

END OF CANDIDATE EDITION